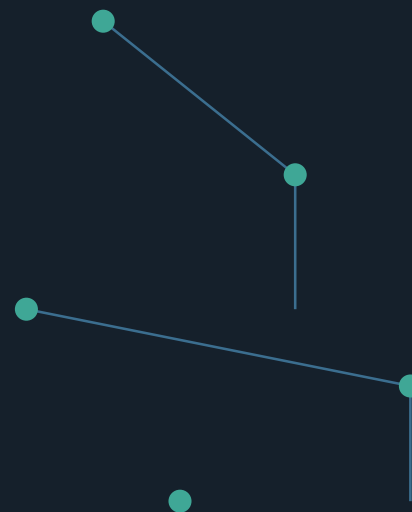


AI 스터디 세미나 · 2026

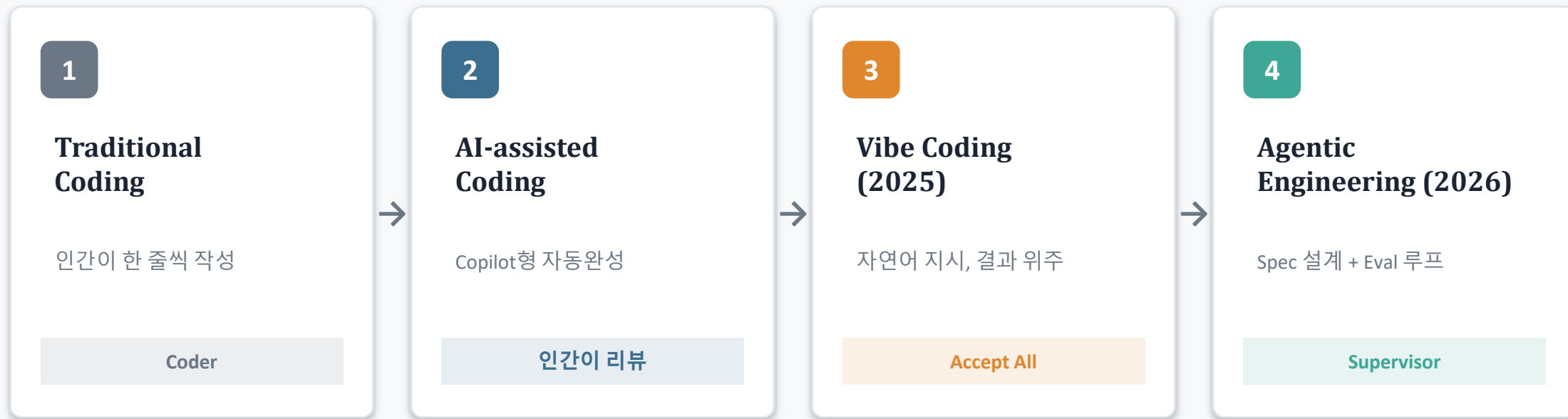
바이트 코딩, 대화에서 실전으로

패러다임의 전환부터, 여섯 단계 실전 루프까지

Part 1 · 왜 바뀌는가 Part 2 · 실전 6단계



개발 패러다임의 4단계 진화



왜 중간 단계가 필요한가 — “AI를 조금이라도 쓰면 바이브 코딩인가?” 라는 질문에 답하려면 AI-assisted 단계를 구분해야 한다. 이 텍스트의 Part 2는 3~4번째 단계를 실무에 붙이는 방법을 다룬다.

Vibe Coding이란?

2025년 2월 2일, Andrej Karpathy가 제안

코드를 직접 검토하지 않고 AI의 제안을 그대로 수용하며(Accept All), 결과와 느낌(Vibe)에 의존해 반복적으로 완성해가는 개발 방식.

(카파시 본인의 원문 취지를 요약·파라프레이즈한 것)

핵심 특징

- 검증 수단이 코드 리뷰 → 결과물을 눈으로 확인(Human Eyeballing)
- 자연어 의도 전달 → 지속적 피드백 루프
- 비개발자도 진입 가능 — 진입장벽을 크게 낮춤

5단계 반복 루프



↳ 만족스러울 때까지 1→5 반복

빠르다는 착각

METR 랜덤화대조실험(RCT) · 2025.7 · 숙련 오픈소스 개발자 16명 · 실제 이슈 246개

—19%

실제 결과 — 더 오래 걸림

초기 2025년 AI 도구 사용 허용 조건

시작 전 기대

+24% 빨라질 것

작업 후 체감

+20% 빨라졌다고 느낌

교훈: 생성량이 아니라 완료 시간과 검증 비용을 측정한다.

※ 특정 개발자·저장소·도구 조건의 RCT이며 전체 개발을 일반화하지 않음 · METR은 2026년 초 후속실험에서 선택편향으로 해석에 한계가 있었다고 밝힘

생산성 뒤에 쌓이는 보안 부채

40%

Stanford

취약한 코드를 만들고도 더 안전하
다고 믿음

6→35

Georgia Tech

AI 생성 코드 관련 CVE, 2026년 1~3
월 (6배)

15/15

Tenzai · CIO.com

생성 앱 전체에서 취약점 발견, 총
69건

43%

슬롭스쿼팅

환각 패키지명이 재실행해도 동일
하게 반복되는 비율

실제 사건

AWS · Kiro가 환경을 삭제/재생성하기로 결정해 13시간 장애 (2025.12)

Moonwell · 오라클 설정 오류로 \$1.8M 배드데트 발생 (2026.2, AI 생성 코드 의심 정황 보도)

“Vibe Coding은 이제 Passé”

Andrej Karpathy · Sequoia Capital AI Ascent · 2026.4.30

	Vibe Coding (2025)	Agentic Engineering (2026)
지향점	바닥(Floor) 높임 — 비개발자도 구축 가능	천장(Ceiling) 높임 — 전문가가 더 큰 시스템 구축
코드 리뷰	거의 안 함 (Diff 미확인, Accept All)	필수 — 정확성·아키텍처 검증
적합 대상	프로토타입, 단기 사이드 프로젝트	상용 프로덕션 시스템
책임 소재	암묵적 — AI가 짠 대로 방치	인간이 명시적으로 책임

이 선언의 의미는 '종말'이 아니라 '한계 인정' — 프로토타입에는 바이브 코딩이 여전히 유효하다.

숙제는 '더 길게 대화하기'가 아니었다

“ 대화로 하지 말고, 도구를 써서 요구사항을 정의하고 설계한 다음, 그 설계대로 코딩하게 하라.

핵심 질문

어떤 도구를 쓰느냐보다, 언제 대화에서 문서와 검증으로 승격할 것인가?

Part 1이 '왜'를 다뤘다면, 지금부터는 그 승격 기준과 실제 작업 순서를 다룬다.

프롬프트 → 코드만 반복하면 속도는 나지만 통제는 사라진다

대화형 바이브

말한다

→ 생성된다

→ 일단 실행한다

빠른 첫 화면에는 강하다

실전형 바이브

의도를 문서화한다

→ 설계 경계를 정한다

→ 작은 단위로 말긴다

→ 증거로 검증한다

운영 가능한 결과물로 닫는다

작고 되돌리기 쉬우면 대화로, 커지고 위험해지면 문서로

대화로 충분

- 한 파일 안의 작은 수정
- 실패해도 바로 되돌릴 수 있음
- 완료 조건이 눈으로 명확함
- 데이터·권한·비용 영향이 낮음

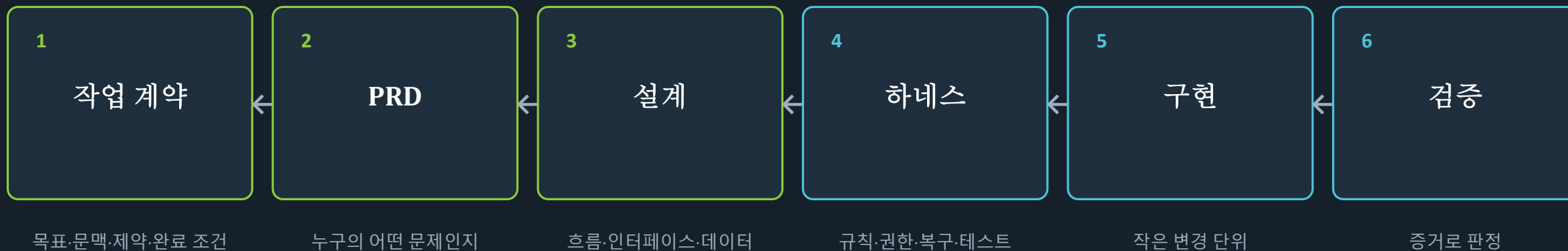
문서와 검증이 필요

- 여러 파일·서비스가 연결됨
- 요구사항이 자주 바뀜
- 인증·결제·개인정보를 다룸
- 다른 사람이 이어받아야 함

승격 기준

범위 × 위험 × 인계 필요성

실전 바이브 코딩은 여섯 단계의 닫힌 루프다



검증 결과가 다음 작업 계약을 갱신한다

6 검증 → 1 작업 계약으로 순환 (오른쪽에서 왼쪽으로 진행)

좋은 시작은 프롬프트가 아니라 '작업 계약'이다

01 목표

무엇이 달라져야 하는가

02 문맥

어디를 읽고 무엇을 보존할 것인가

03 제약

하지 말아야 할 변경은 무엇인가

04 완료 조건

무엇을 보여주면 끝인가

예시

목표

로컬 LLM에 질문하는 POST /chat 구현

문맥

기존 FastAPI 구조와 JSON 응답 유지

제약

새 프레임워크 추가 금지, 타임아웃 15초

완료 조건

테스트 통과 + 한국어 응답 + 오류 예시

PRD는 길게 쓰는 문서가 아니라 판단을 고정하는 한 장이다

사용자	누가 언제 쓰는가
문제	현재 무엇이 불편한가
핵심 흐름	사용자가 어떤 순서로 성공하는가
범위	이번에 반드시 포함할 것
제외	이번에는 만들지 않을 것
인수 조건	동작을 판정할 테스트 문장

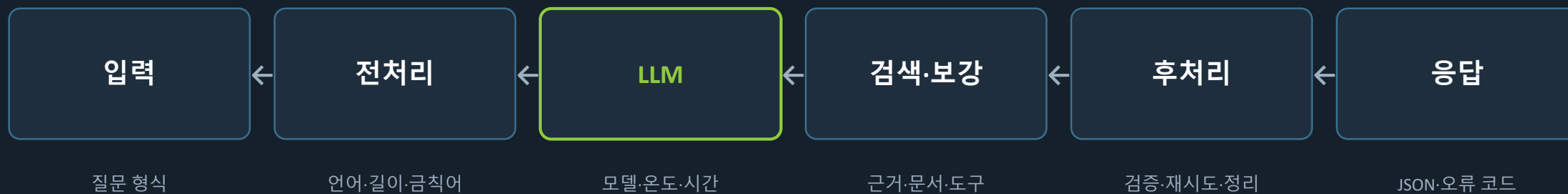
좋은 PRD의 기준

AI가 읽었을 때
추측할 빈칸이 줄고

사람이 읽었을 때
포기할 범위가 보이며

테스트가 읽었을 때
성공·실패가 갈린다

설계는 '코드 모양'보다 입력과 출력의 경계를 먼저 그린다

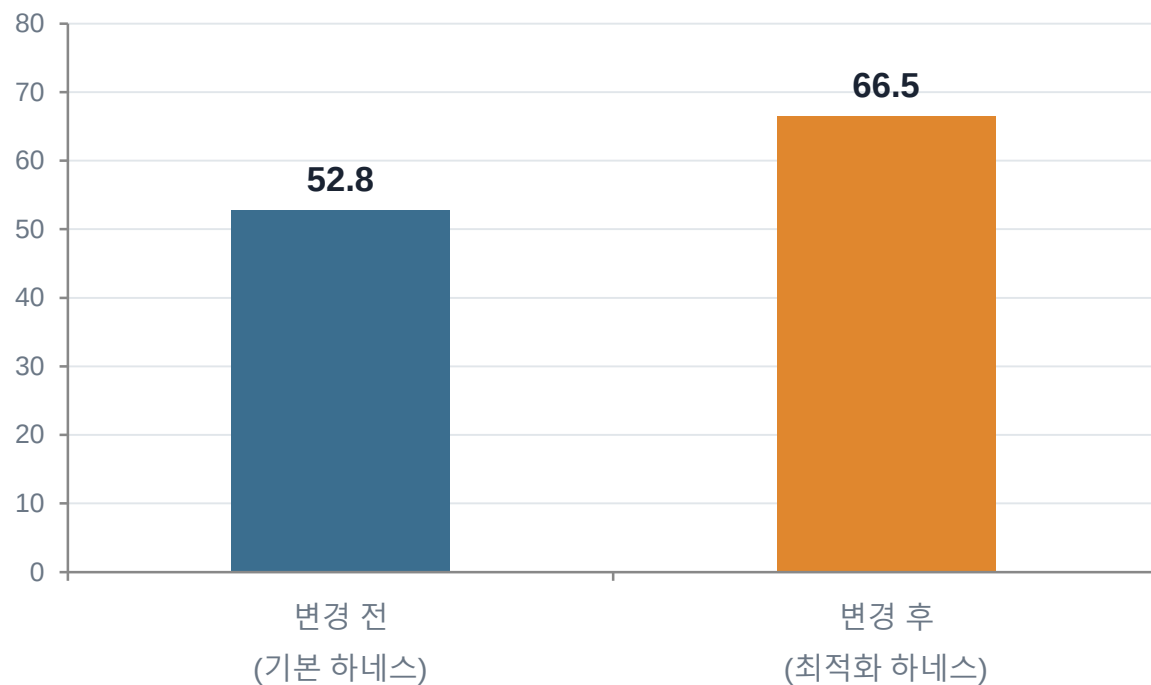


이 흐름 하나가 체인이고, 여러 체인과 자원·시간·권한을 제어하면 오케스트레이션이 된다.

모델이 아니라 하네스가 성능을 가른다

LangChain 자사 블로그, 2026.2.17 — 모델(GPT-5.2-Codex) 고정, 하네스만 교체

TerminalBench 2.0 점수



순위 변화

Top 30권 → Top 5

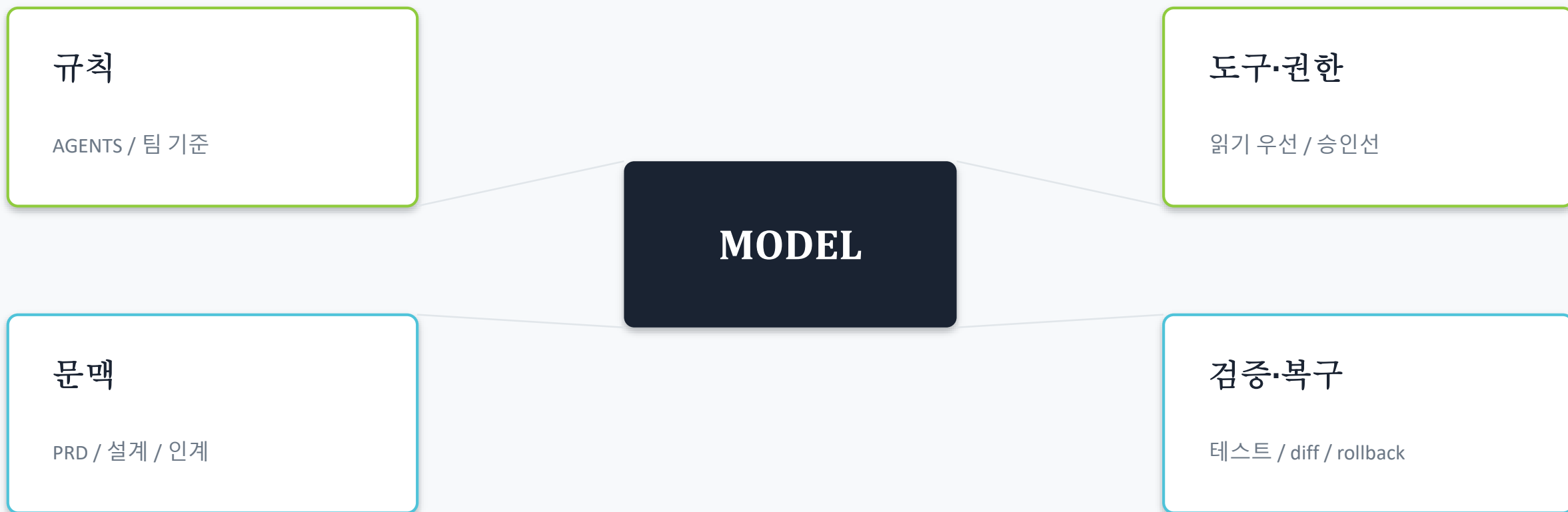
+13.7%p 상승 — 모델은 동일, 하네스만 변경

Vercel 툴 80% 제거 → 성공률 80%→100%

Princeton CORE-Bench 동일 모델, 스캐폴드만 다르게
42%→78%

모델을 교체해 5% 개선하는 것보다, 하네스를 설계해 15% 개선하는 것이 현실적이다.

하네스는 모델이 아니라 '일하는 환경'을 설계하는 일이다



하네스가 강할수록 더 많이 맡기면서도 덜 불안하다.

도구는 서열이 아니라 역할로 섞는다

탐색·정리

ChatGPT · Gemini

자료 비교, 질문 구체화, PRD 초안

설계·시각화

문서 도구 · Canva

흐름도, 스토리보드, 발표 자료

저장소 구현

Codex · Claude Code · Cursor

파일 탐색, 수정, 테스트, diff

검증·합류

CLI · CI · GitHub

재현, 자동 테스트, 리뷰, 되돌리기

한 도구에 끝까지 버티지 말고, 질문이 바뀌면 작업 창구를 바꾼다.

검증은 AI에게 '잘했니?'라고 다시 묻는 일이 아니다

1. 결정적 검사

형식·숫자·기한·스키마
테스트·lint·타입·쿼리 결과

2. 동작 관찰

실제 API 호출·화면·로그
오류·타임아웃·경계값

3. 사람 판단

diff·보안·설계 의도
배포·승인·책임

LLM은 설명과 후보 생성에 강하고, 판정은 가능한 한 코드와 증거에 맡긴다.

세미나 사례: 숫자·기한은 즉시 검증, 설명은 LLM 활용

AI의 답을 전달하지 말고, 자기 판단을 전달한다

고기 프록시

AI가 만든 코드·문서를 이해 없이 다른 사람에게 그대로 넘기는 인간 전달자

복사 → 전달 → 책임 전가

“AI가 이렇게 말했습니다”

소화 → 검증 → 재구성

- 1 무엇을 말했는지 말한다
- 2 무엇으로 검증했는지 보여준다
- 3 남은 위험을 자기 언어로 설명한다
- 4 최종 선택과 책임은 사람이 진다

실습: 로컬 LLM API를 '문서부터' 만든다

과제

POST /chat 하나를 만들고, 실패해도 설명 가능한 상태로 닫기

- ① 작업 계약 4줄
- ② 1페이지 PRD
- ③ 체인 설계도
- ④ 코드 diff
- ⑤ 테스트 증거

힌트: 처음에는 챗봇 전체가 아니라 '문자열 입력 → 응답 반환'부터

완료 조건

- 질문을 JSON으로 받는다
- 로컬 모델을 호출한다
- 한국어 응답과 길이를 보장한다
- 15초 타임아웃과 오류 응답이 있다
- 정상·오류 테스트가 모두 통과한다
- 변경 이유와 남은 위험을 설명한다

마지막으로 세 문장만 남긴다

무엇을 맡길 것인가

무엇으로 검증할 것인가

무엇을 끝까지 책임질 것인가

바이브 코딩의 실력은 많이 생성하는 능력이 아니라,
정확히 말기고 싸게 검증하고 책임 있게 달는 능력이다.

Coder → Specifier / Supervisor — Part 1 에서 다룬 역할 전환이, 지금 이 여섯 단계의 실전 근거다.